Nirgude Rohit Vishwas, 24CS01052, Group3

GPU Model: Nvidia GeForce RTX 4050 Laptop GPU
Compute Capability: 8.9

Question 1: Shared Memory Capacity Evaluation. Write a CUDA kernel using static shared memory allocation.

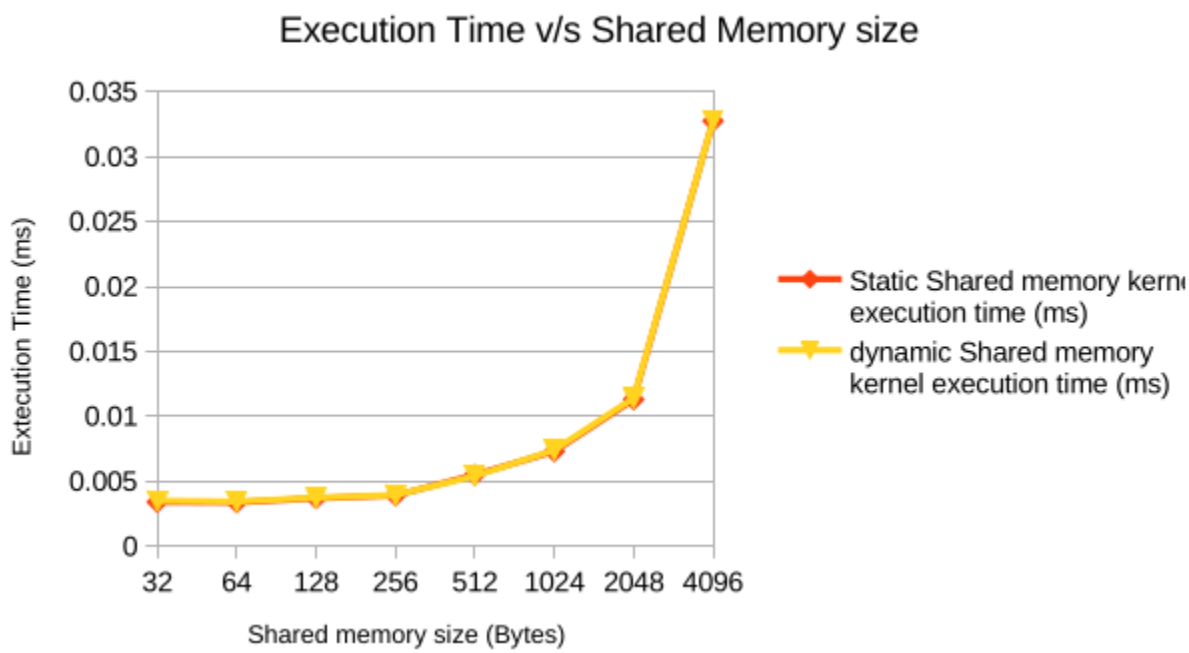
	Requested Shared Memory Size (Bytes)	Compile Status	Run status
2^5	32	success	success
2^6	64	success	success
2^7	128	success	success
2^8	256	success	success
2^9	512	success	success
2^10	1024	success	success
2^11	2048	success	success
2^12	4096	success	success
2^13	8192	success	success
2^14	16384	success	success
2^15	32768	success	success
	49152	success	success
	49156	success	Failure
2^16	65536	success	Failure
2^17	131072	success	Failure
2^18	262144	success	Failure
2^19	524288	success	Failure
2^20	1048576	success	Failure
2^21	2097152	success	Failure
2^22	4194304	success	Failure
2^23	8388608	success	Failure
2^24	16777216	success	Failure
2^25	33554432	success	Failure
2^26	67108864	success	Failure
2^27	134217728	success	Failure
2^28	268435456	success	Failure
2^29	536870912	success	Failure
2^30	1073741824	success	Failure
2^31	2147483648	Success	Failure
	4294967292	Success	Failure
2^32	4294967296	Failure	Failure
2^33	8589934592	Failure	Failure

Max for compilation	4294967292 Bytes
Max for Launch	49152 Bytes

Shared Memory Per Block	Shared Memory Per Multiprocessor
49152 Bytes	102400 Bytes
From CUDA Runtime API	

Question 2: Static vs Dynamic Shared Memory Benchmark

Shared Memory size (Bytes)	Static Shared memory kernel execution time (ms)	dynamic Shared memory kernel execution time (ms)
32	0.0034	0.0035
64	0.00337	0.00343
128	0.00369	0.00374
256	0.0039	0.00394
512	0.0055	0.0054
1024	0.0073	0.0074
2048	0.0113	0.0114
4096	0.032751	0.032776

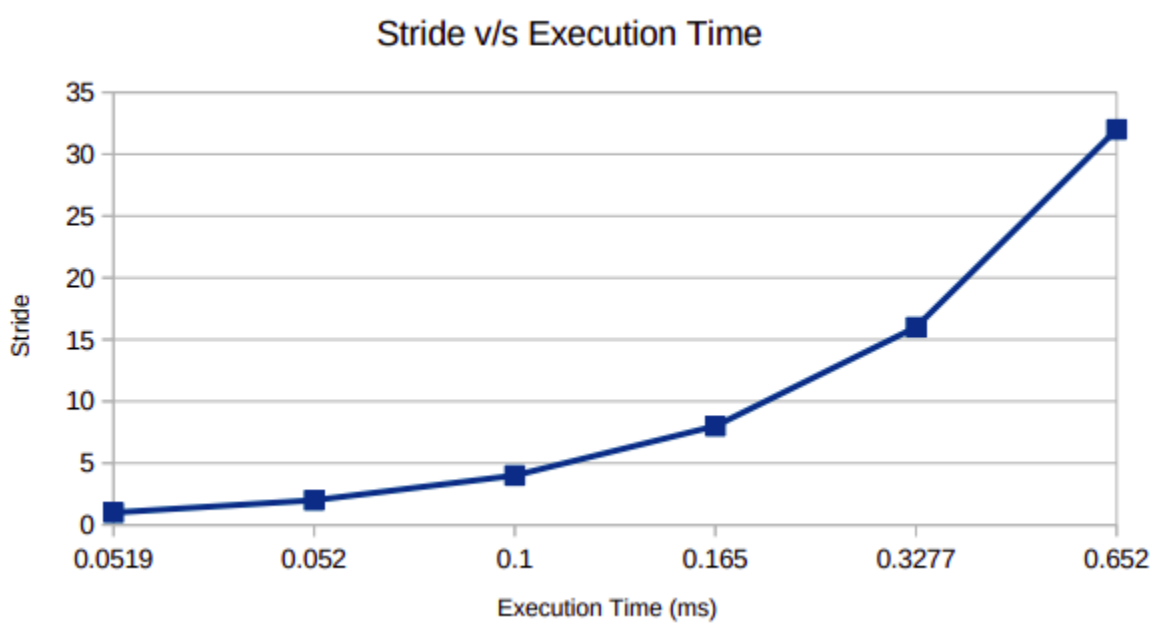


Explanation: Here, 2 kernels have been implemented, one of them uses static shared memory and other one uses dynamic shared memory and performs the same task. We can observe that statically memory allocated kernel runs slightly faster than the dynamically allocated one, this is because in case of dynamic shared memory there is an overhead of the memory size and allocation during the runtime which takes a bit more time, where as in case of static shared memory the compiler already knows the memory size and allocates it during the compile time which saves time during the runtime as the memory was already allocated.

Question 3: Shared Memory Access Pattern & Bank Conflict Study. Implement a shared memory kernel with two access patterns:

Stride	Runtime (ms)	Relative Slow Down	
1	0.0519	1.00	No bank Conflicts (Sequential)
2	0.052	1.00	2 way Bank conflicts
4	0.1	1.93	4 way Bank conflicts
8	0.165	3.18	8 way Bank conflicts
16	0.3277	6.31	16 way Bank conflicts
32	0.652	12.56	32 way Bank conflicts

Relative SlowDown = runtime(stride)/runtime(stride=1)



Explanation: The kernel defines shared memory as an int array with 1024 elements, now since this shared memory is stored in 32 banks. Here the element with index i, where i%32==0 will be stored in bank 0, then where i%32==1 will be stored in bank 1 and so on.

The kernel is launched with 32 threads per block, now when stride = 1, all 32 threads will access shmem[threadIdx], so one element is accessed by one thread in one bank, therefore no bank conflicts. When stride=2, only half of the banks are used and 2 threads map to the same Banks accessing 2 different elements causing a 2way bank conflict. When stride = 4, for banks with id i, where i%4==0, 4 threads will access 4 different elements in the same bank causing a 4 way bank conflict and slowing down the performance and increasing execution time. In the same way for stride = 8,16,32, there will be a 8way bank conflict, 16 way bank conflict and 32 way bank conflict giving the poorest performance and max execution time.